MODÉLISATION D'UN BRAS ROBOTIQUE:

URDF / Xacro & MoveIt 2

Dhunnoo Yashta

13 Jun 2026

Table of Contents

1. Introduction 2
2. Choice of Representation 2
3. Robot Modelling 3
3.1 Robot Structure 3
3.2 Joints 3
3.3 Visual Elements 4
3.4 Collision Models 4
3.5 Inertial Properties 4
3.6 Joint Limits 5
4. Verification in RViz 2 5
5. MoveIt 2 Configuration 8
5.1 Planning Groups 8
5.2 End Effector 9
5.3 Kinematics Configuration 9
5.4 Controller Configuration 9
5.5 Motion Planning Validation 9
6. ROS 2 Package Structure 12
7. Difficulties Encountered and Solutions 12
8. GitHub 13
9. Conclusion 13
10. References 14

1. Introduction

The objective of this project was to model a robotic arm using ROS 2 and integrate it with MoveIt 2 for motion planning and simulation. The robot was provided as STL mesh files together with information about the position and orientation of each component. The final goal was to create a complete robot description, visualize the robot in RViz 2, and perform trajectory planning using MoveIt 2.

The project was developed using ROS 2 Jazzy, URDF/Xacro, MoveIt 2, RViz 2 and ROS 2 Control.The final result allows the robot arm and gripper to perform motion planning and trajectory execution successfully inside the ROS 2 Jazzy environment.

2. Choice of Representation

Two possible approaches were available:

- A single URDF file
- A modular Xacro architecture

For this project, the Xacro approach was selected.

Reason for Choosing Xacro

The robotic arm contains several links, joints, controller configurations and reusable parameters. Managing all these elements in a single URDF file would make the code difficult to read and maintain. Xacro allows the robot description to be separated into multiple files, making the project more organized.

Advantages of Xacro

1. Easier to read and maintain
2. Supports reusable macros
3. Allows separation of robot description and controller configuration
4. Simplifies future modifications

Disadvantages of Xacro

1. Slightly more complex than a simple URDF
2. Requires Xacro processing before generating the final URDF

A simple URDF would be sufficient for a very small robot with only a few links and joints. Since this project contains multiple components and configurations, Xacro was the better choice.

3. Robot Modelling

3.1 Robot Structure

The robot consists of the following main links:

- base_link
- base_plate
- forward_drive_arm
- horizontal_arm
- claw_support
- gripper_left
- gripper_right

The structure follows the linkage diagram provided in the assignment.

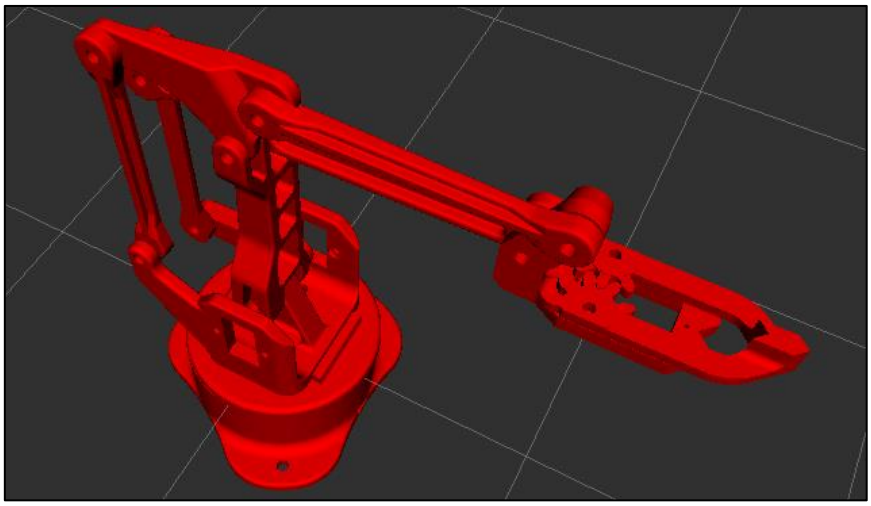

Figure 1: Assembly of STL Parts

3.2 Joints

The robot contains the following joints:

Joint	Type	Function
joint1	Revolute	Base rotation
joint2	Revolute	Arm lifting motion
joint3_vertical_drive_arm	Passive	Mechanical linkage
joint4_triangular_link	Passive	Mechanical linkage
joint5_left_gear	Revolute	Gripper motion
joint6_right_gear	Mimic	Opposite gripper motion

The right gripper was configured as a mimic joint to automatically follow the movement of the left gripper: <**mimic** joint="joint5_left_gear" multiplier="-1"/>

3.3 Visual Elements

The STL meshes supplied with the assignment were used for visualization.

A scale factor of:

scale="0.01 0.01 0.01"

was applied because the original meshes were larger than the required size.

The mesh positions and orientations were implemented according to the coordinates provided in the assignment document.

3.4 Collision Models

Collision geometries were added to all links.

Simple geometric shapes were used to reduce computational complexity:

- Cylinders for the base
- Boxes for arm sections
- Small boxes for gripper components

Using simplified collision models improves planning performance while maintaining acceptable accuracy.

3.5 Inertial Properties

Estimated masses and inertia values were assigned to all links.

Since the real mechanical properties were not provided, reasonable approximations were used.

Example assumptions:

Component	Estimated Mass
Base	1.0 kg
Arm Links	0.5 kg
Gripper Parts	0.1 kg

These values were sufficient for simulation purposes.

3.6 Joint Limits

Joint limits were defined for all movable joints.

Example:

Joint	Lower Limit	Upper Limit
joint1	-3.14 rad	3.14 rad
joint2	-1.57 rad	1.57 rad
joint5_left_gear	0 rad	1.309 rad

Velocity and acceleration limits were also configured to allow MoveIt 2 trajectory generation.

4. Verification in RViz 2

The robot model was tested in RViz 2.

The following verifications were successfully completed:

- Robot model displayed correctly
- STL meshes loaded correctly
- TF tree generated without errors
- Joint states published correctly
- Robot motion visible in RViz 2

The robot could be manipulated through the Motion Planning plugin and the joint state publisher.

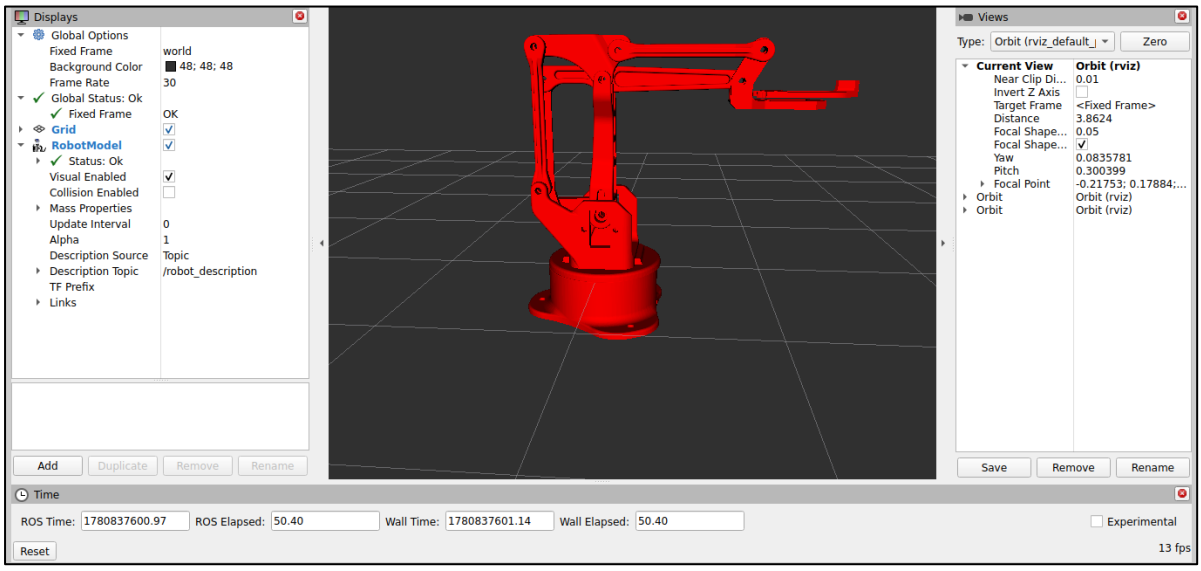

Figure 2: Robot model displayed correctly

Below generated TF tree confirmed that all robot links were connected correctly and no TF errors were present. The hierarchy followed the robot kinematic chain from world to the gripper links.

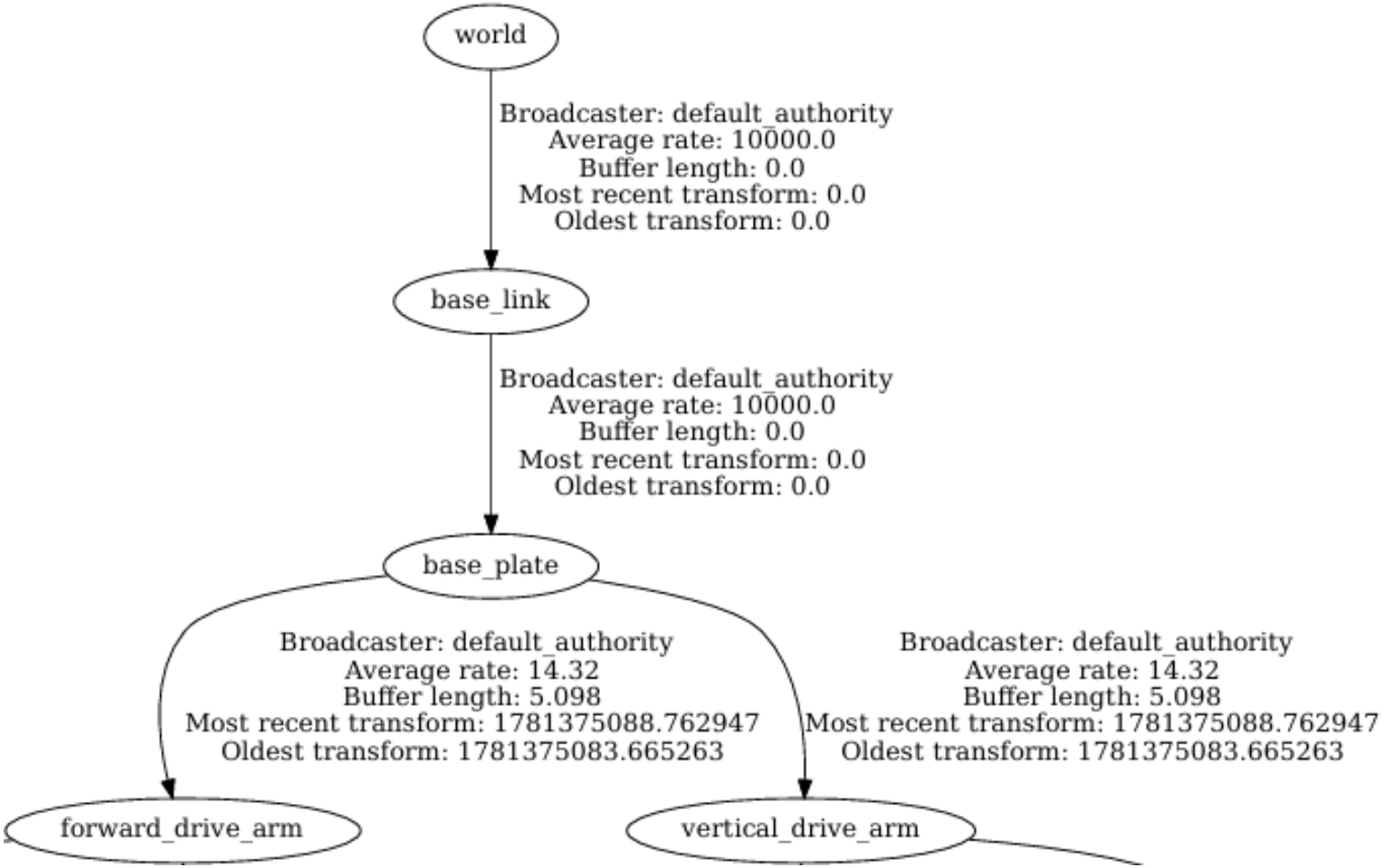

Figure 3: TF Tree - Part 1

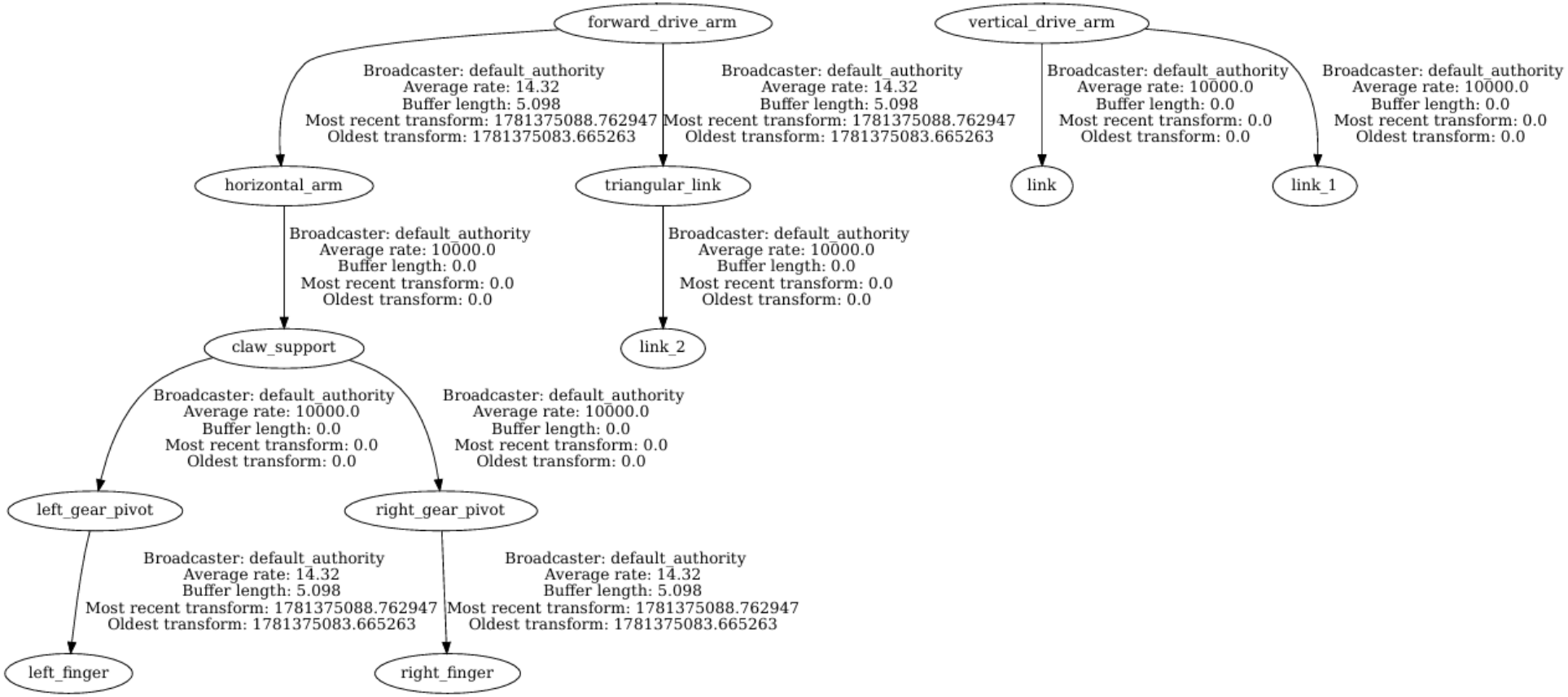

Figure 4: TF Tree - Part 2

5. MoveIt 2 Configuration

MoveIt Setup Assistant was used to generate the configuration package.

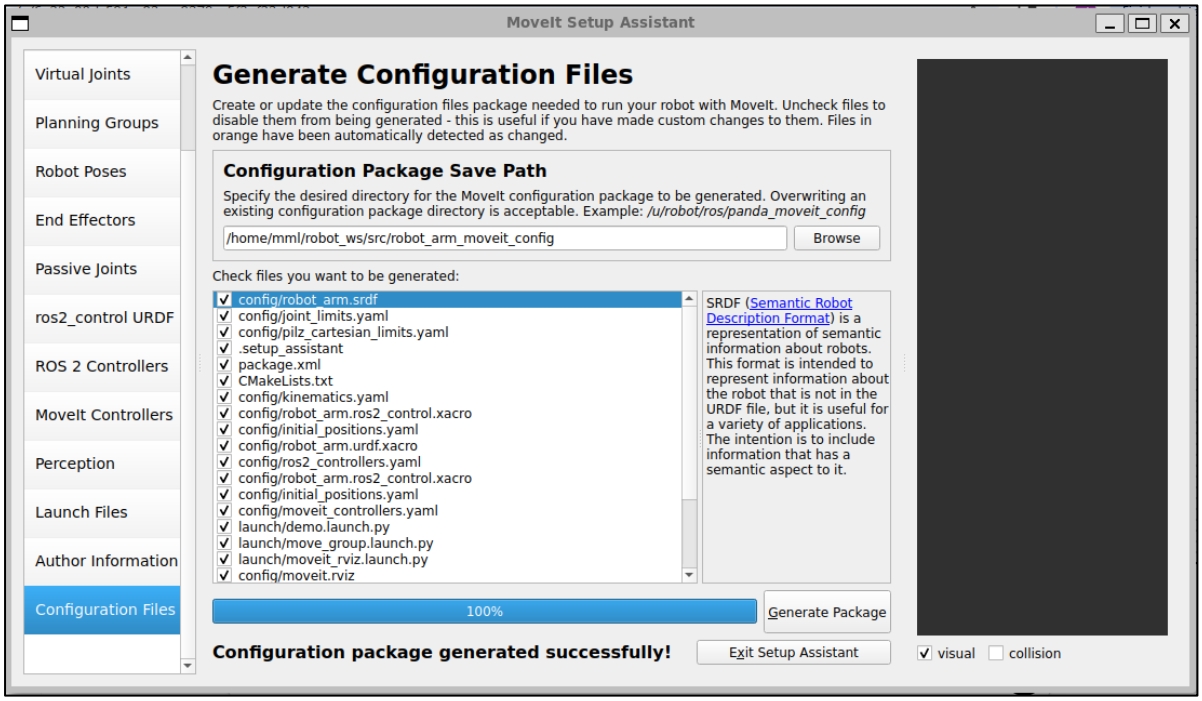

Figure 5:Configuration package successfully generated on MoveIt 2

5.1 Planning Groups

Two planning groups were created:

1. **Arm Group**

   It contains:

   - joint1
   - joint2

   This group controls the robot arm motion.

2. **Gripper Group**

   Contains:

   - joint5_left_gear

   This group controls the gripper opening and closing motion.

5.2 End Effector

The gripper was configured as the robot end-effector. It allows MoveIt 2 to recognize the gripper as the tool used for manipulation tasks.

5.3 Kinematics Configuration

A kinematics solver was configured for the arm planning group. It enabled inverse kinematics calculations required for motion planning.

5.4 Controller Configuration

Two trajectory controllers were configured:

1. **arm_controller**

   It controls:

   - joint1
   - joint2

2. **gripper_controller**

   It controls:

   - joint5_left_gear

   The joint_state_broadcaster was also enabled to publish robot joint states.

5.5 Motion Planning Validation

Several tests were performed.

Test 1: Arm Motion

Start State: home

Goal State: raised

Result:

- Planning successful
- Execution successful

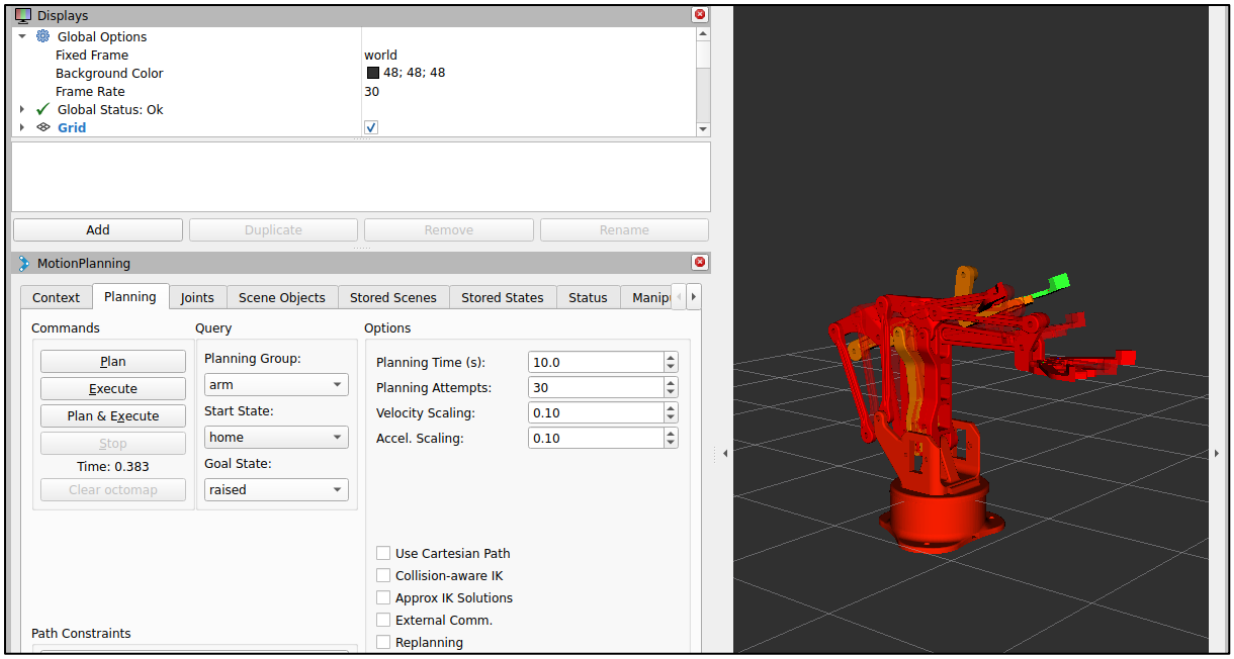

Figure 6: Arm motion planned successfully

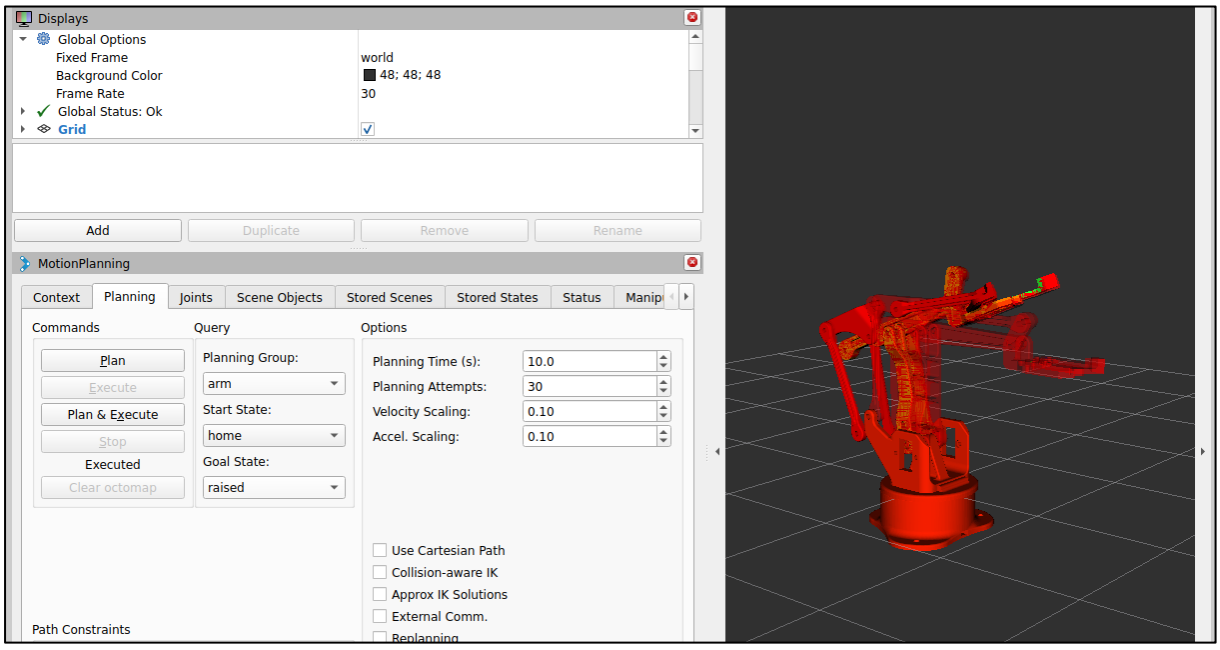

Figure 7: Arm motion executed successfully

Test 2: Gripper Motion

Start State: closed

Goal State: open

Result:

- Planning successful
- Execution successful

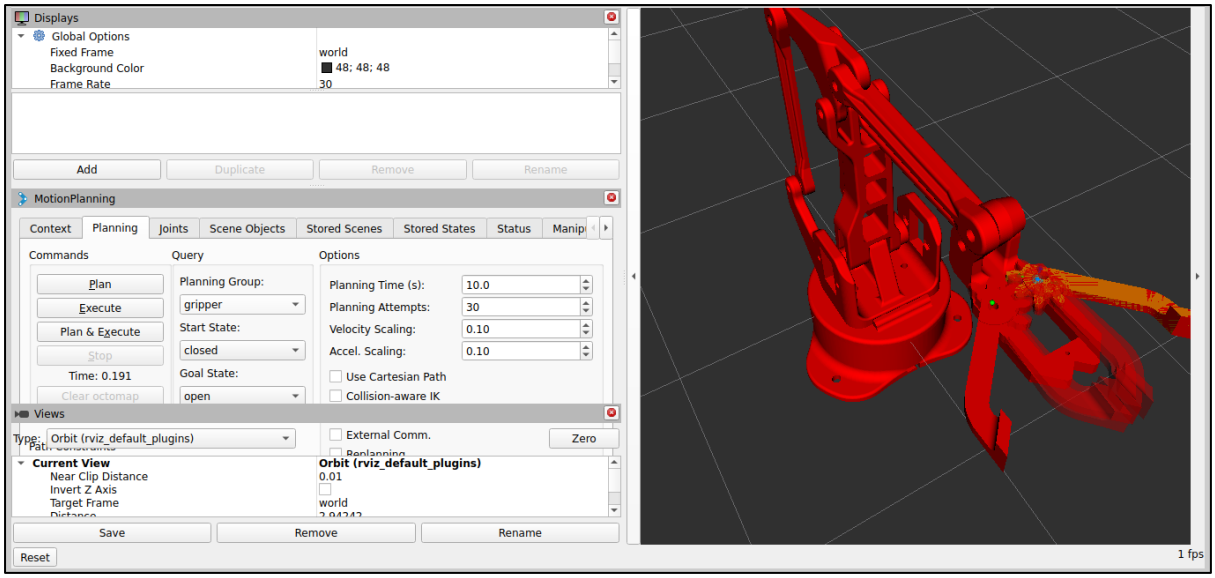

Figure 8: Gripper motion planned successfully

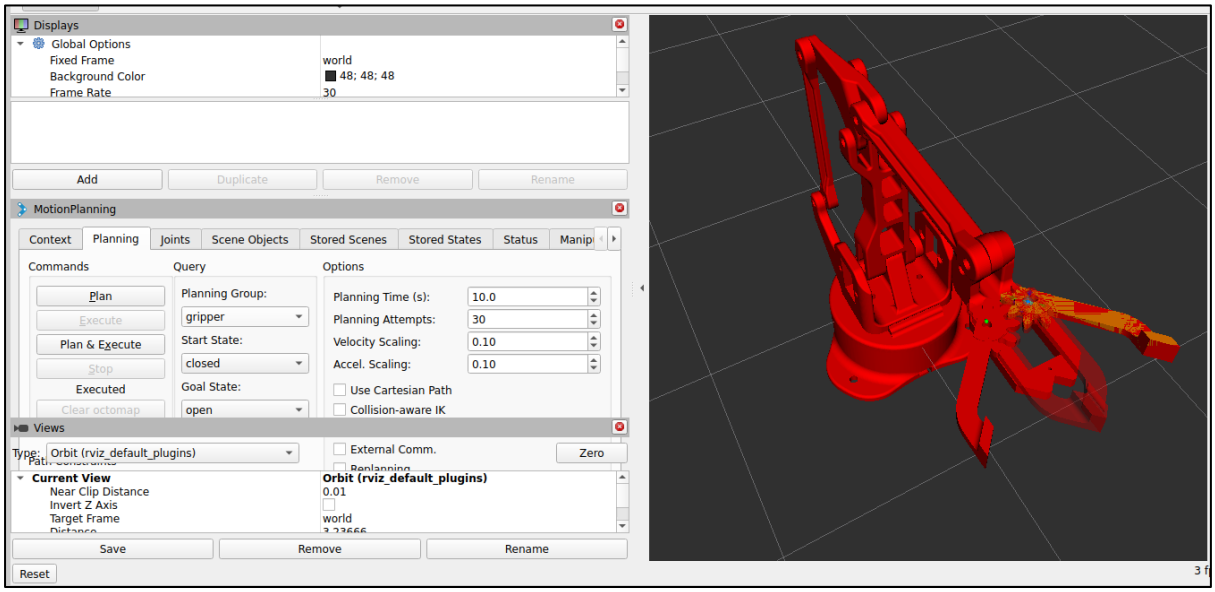

Figure 9: Gripper motion executed successfully

The final system was capable of planning and executing trajectories for both the robotic arm and the gripper.

6. ROS 2 Package Structure

The project follows the ROS 2 package organization required by the assignment.

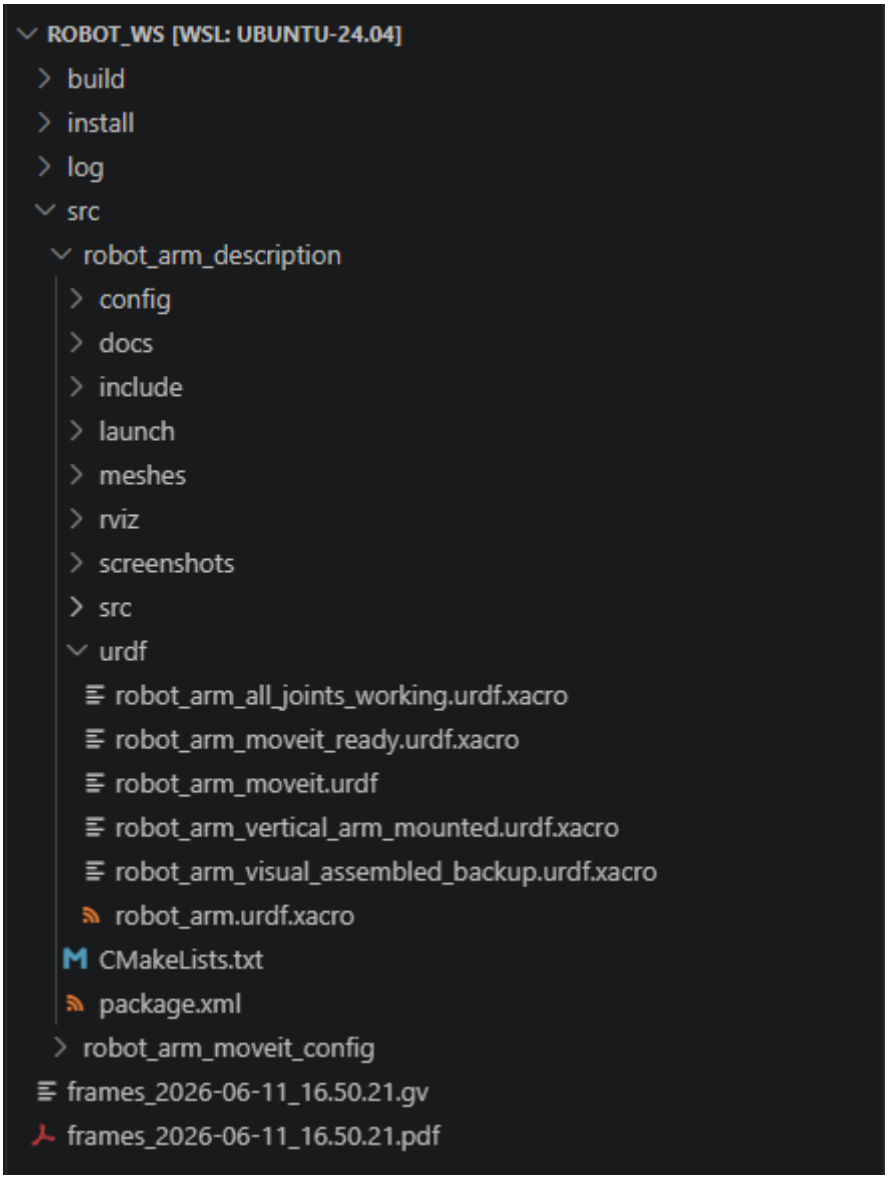

Figure 10: ROS 2 Package Structure

7. Difficulties Encountered and Solutions

Several issues were encountered during development.

Problem 1: MoveIt Could Not Find Kinematics

Error:

No kinematics plugins defined

Solution:

A valid kinematics.yaml file was created and linked to the planning groups.

Problem 2: Planning Failed

MoveIt reported planning failures.

Cause:

- Missing acceleration limits
- Incorrect controller configuration

Solution:

Joint acceleration limits were added and controller parameters were corrected.

Problem 3: Controllers Did Not Execute Trajectories

MoveIt generated plans but the robot did not move.

Cause:

Incorrect ROS 2 Control configuration.

Solution:

The ros2_control configuration was updated and the GenericSystem hardware plugin was used.

8. GitHub

https://github.com/YashtaD/robot-arm-moveit2-jazzy

9. Conclusion

This project successfully demonstrated the complete workflow of modelling a robotic arm using Xacro, integrating it with ROS 2 Control, configuring MoveIt 2 and validating trajectory planning in RViz 2.

The final robot model loads correctly, generates valid trajectories and executes arm and gripper motions successfully. The project therefore satisfies the objectives defined in the assignment and provides a foundation for future work involving Gazebo simulation or real hardware implementation.

10. References

1. MoveIt Documentation, 2026. *MoveIt 2 Documentation*. Available at: MoveIt Documentation [Accessed 10 June 2026].
2. Open Robotics, 2026. *ROS 2 Documentation (Jazzy Jalisco)*. Available at: ROS 2 Documentation [Accessed 09 June 2026].
3. Open Robotics, 2026. *ROS 2 Design Documentation*. Available at: ROS 2 Design Documentation [Accessed 11 June 2026].
4. PickNik Robotics, 2026. *MoveIt Motion Planning Framework*. Available at: MoveIt Official Website [Accessed 10 June 2026].